SQL Explanation Grader: baseline vs RAG

Experiment **rag_vs_baseline_dev** · split **dev** (50 items) · generator **llama3.2:3b** · judge **qwen2.5:7b** (rubric) · generated 2026-10-26 18:08

How to read this report. "Judge approval" is the share of explanations an LLM judge graded good. It is an automated, unverified measurement, not ground truth. Deterministic checks are rule-based heuristics computed from the SQL structure. Agreement of the judge with reference labels is reported separately (results/rag/judge_agreement.json).

Headline comparison

Metric	Baseline (prompt only)	RAG
Judge approval (95% CI)	82.0% (41/50) [69-90]	60.0% (30/50) [46-72]
Approval, abstentions count as fail	82.0% (41/50) [69-90]	60.0% (30/50) [46-72]
Rubric pass (judge's own scores)	70.0% (35/50) [56-81]	52.0% (26/50) [39-65]
Mean correctness (1-5)	4.08	3.68
Mean completeness (1-5)	4.42	3.98
Mean clarity (1-5)	3.76	3.30
Mean groundedness (1-5)	4.42	3.52
Explanations with unsupported claims (judge)	18.0% (9/50) [10-31]	28.0% (14/50) [17-42]
Deterministic check coverage	0.96	0.98
Invented numbers (rule-based)	10.0% (5/50) [4-21]	12.0% (6/50) [6-24]
Raw SQL keywords in text	10.0% (5/50) [4-21]	66.0% (33/50) [52-78]
Abstentions	0.0% (0/50) [0-7]	0.0% (0/50) [0-7]
Generation errors	0	0
Mean end-to-end latency (s)	6.6	17.5
Mean retrieval time (s)	0.00	1.49
Mean input tokens	571	2647
Mean output tokens	104	266

AI-assisted reference labels (not human): share labelled good

	Baseline	RAG
Labelled good	20/50	10/50
Paired: only baseline good / only RAG good	16	6
RAG minus baseline (95% bootstrap CI)	-20.0 pts (-38.0 to -2.0)	McNemar p = 0.052

Labels and their decision rules: data/labels/. They are not produced by the judge in this report.

Paired comparison (same items, both judged)

50 pairs. Both good: 26; both bad: 5; only baseline good: 15; only RAG good: 4. Approval difference RAG minus baseline: -22.0 points (95% bootstrap CI -38.0 to -6.0). Exact McNemar p = 0.0192.

Dimension	Mean RAG minus baseline	95% CI
correctness	-0.40	-0.84 to +0.02
completeness	-0.44	-0.76 to -0.14
clarity	-0.46	-0.70 to -0.22
groundedness	-0.90	-1.44 to -0.40

Grounding and retrieval (RAG condition)

Metric	Value
Answers with at least one citation	90.0% (45/50) [79-96]
Citation validity (cited source was provided)	1.000
Answers citing a source that was not provided	0.0% (0/50) [0-7]
Share of sentences carrying a citation	0.447
Lexical support of cited sentences (heuristic)	0.806
Model said evidence was insufficient	0
Retrieval feature coverage	1.000
Evidence judged sufficient before generation	100.0% (50/50) [93-100]
Grounding status counts	{"grounded": 45, "weakly_grounded": 5}

Retrieval evaluation (no LLM involved)

Project-authored by Claude (AI-assisted) on 2026-10-26 by reading the section headings of each corpus document. Not human-reviewed yet. Treat retrieval scores computed with it as rule-based estimates, not human relevance judgements.

Chunk	Strategy	Mode	MRR	Recall@5	Prec@5	nDCG@5	Ctx recall	ms/query
400	decomposed	dense	0.824	0.925	0.556	0.616	0.892	161
400	decomposed	bm25	0.642	0.836	0.568	0.560	0.960	5
400	decomposed	hybrid	0.690	0.867	0.585	0.590	0.893	162
400	probe	hybrid	0.564	0.596	0.397	0.417	0.711	50
400	raw_sql	hybrid	0.691	0.797	0.522	0.538	0.799	58
200	decomposed	dense	0.784	0.899	0.571	0.584	0.943	161
200	decomposed	bm25	0.710	0.837	0.637	0.627	0.963	7
200	decomposed	hybrid	0.640	0.878	0.604	0.569	0.948	163
300	decomposed	dense	0.764	0.676	0.511	0.548	0.934	162
300	decomposed	bm25	0.658	0.842	0.597	0.576	0.962	5
300	decomposed	hybrid	0.644	0.834	0.535	0.531	0.940	160
500	decomposed	dense	0.694	0.727	0.448	0.500	0.873	160
500	decomposed	bm25	0.728	0.839	0.589	0.604	0.968	4
500	decomposed	hybrid	0.647	0.854	0.566	0.564	0.874	166
600	decomposed	dense	0.743	0.645	0.458	0.532	0.932	162
600	decomposed	bm25	0.705	0.835	0.548	0.571	0.913	6
600	decomposed	hybrid	0.615	0.830	0.490	0.502	0.899	160

Breakdown by SQL feature (dataset label)

Feature	n	Baseline approval	RAG approval	Base checks	RAG checks
GROUP BY with HAVING	8	75.0% (6/8) [41-93]	100.0% (8/8) [68-100]	0.88	0.88
INNER JOIN of two tables	9	88.9% (8/9) [56-98]	33.3% (3/9) [12-65]	1.00	1.00
JOIN of three tables with ORDER BY and LIMIT	8	62.5% (5/8) [31-86]	0.0% (0/8) [0-32]	0.88	1.00
LEFT JOIN with IS NULL filter	3	100.0% (3/3) [44-100]	66.7% (2/3) [21-94]	1.00	1.00
simple SELECT with WHERE	5	100.0% (5/5) [57-100]	100.0% (5/5) [57-100]	1.00	1.00
subquery in WHERE	10	70.0% (7/10) [40-89]	80.0% (8/10) [49-94]	1.00	1.00
window function such as ROW_NUMBER or RANK	7	100.0% (7/7) [65-100]	57.1% (4/7) [25-84]	1.00	1.00

Items where the two conditions were graded differently

q001 · window function such as ROW_NUMBER or RANK

```
SELECT RANK() OVER (ORDER BY order_date) AS order_rank FROM orders;
```

Baseline: good
This query calculates a ranking of the orders based on their order date. The ranking is assigned to each order, with the earliest order getting a rank of 1, the next earliest order getting a rank of 2, and so on. The result is a list of orders with their corresponding ranking, where the order with the earliest order date has a rank of 1, the order with the next earliest order date has a rank of 2, and so on. The ranking is not based on any specific product or customer, but rather on the order date itself.
Judge: The explanation accurately describes the query, including the use of the RANK() window function and the ordering by order_date. It is clear and grounded in the provided schema.

RAG: bad
This query ranks orders by their order date, assigning a unique rank to each distinct order date. The rank is calculated by the row_number() function, which assigns a number to each row within its partition (i.e., rows with the same order date), and then the rank() function assigns a rank to each row within its peer group (i.e., rows with the same order date and quantity). [S3]
Judge: The explanation incorrectly states that row_number() and rank() are used, when only rank() is in the query. The query does not group by order date or quantity, so the rank is based on the order date only. The explanation also invents the business purpose of ranking orders by date, which is not supported by the schema.

---

q009 · window function such as ROW_NUMBER or RANK

```
SELECT *, RANK() OVER (ORDER BY balance DESC) FROM accounts;
```

Baseline: good
This query selects all columns from the accounts table and assigns a ranking to each account based on its balance in descending order. The ranking starts from 1 for the account with the highest balance, and each subsequent account is assigned a rank one more than the previous one. For example, if the account with the highest balance has a balance of 1000, the next account with a balance of 900 would be ranked 2, the next account with a balance of 800 would be ranked 3, and so on. The ranking is not based on any specific account, but rather on the balance of each account in the table. The query does not return any data, but rather assigns a ranking to each account based on its balance.
Judge: The explanation is correct and complete, but could be more clear in translating SQL terms to plain language.

RAG: bad
This query ranks the clients by their account balance in descending order. The RANK() function assigns a rank to each row within its partition, with the highest balance getting rank 1, the next highest balance getting rank 2, and so on. The rows are numbered starting from 1 in the order defined by the ORDER BY clause in the window definition, or in arbitrary order otherwise. [S1]
Judge: The explanation is incorrect because the query ranks accounts, not clients. It also incorrectly states that the rows are numbered starting from 1 in an arbitrary order, when they are ordered by balance. The explanation speculates about clients and arbitrary order, which are unsupported by the SQL and schema.

---

q012 · window function such as ROW_NUMBER or RANK

```
SELECT *, ROW_NUMBER() OVER (ORDER BY amount) FROM transactions;
```

Baseline: good
This query selects all columns from the transactions table and assigns a unique number to each row based on the amount in descending order. The number is calculated using a window function that looks at the amount of each row and assigns a sequential number starting from 1, with higher amounts getting lower numbers. For example, if the amounts are 100, 200, and 50, the query will assign the numbers 1, 2, and 3, respectively, to the rows in that order.
Judge: The explanation is correct, complete, and clear, and all claims are supported by the SQL and schema.

RAG: bad
This query selects all columns from the transactions table and assigns a unique number to each row within its partition, ordered by the amount column. The row numbers are assigned starting from 1, and if there is no ORDER BY clause, all rows are considered peers and the function always returns 1. [S2]
Judge: The explanation incorrectly states that the ORDER BY clause is missing, and it incorrectly describes the behavior of ROW_NUMBER() without an ORDER BY clause. The explanation also does not cover the actual schema or the tables involved.

---

q014 · GROUP BY with HAVING

```
SELECT title, AVG(year) FROM books GROUP BY title HAVING AVG(year) > 2010;
```

Baseline: bad
This query selects the titles of books that were published after 2010. It calculates the average publication year for each book title and only returns the titles where the average year is greater than 2010. The result is a list of book titles with their corresponding average publication year, but only for books published after 2010.
Judge: The explanation incorrectly states that the query filters books published after 2010, but the HAVING clause filters based on the average year, not the individual book years. Also, the query does not return a list of book titles with their average publication year, but rather a single value for the average year of books that meet the condition.

RAG: good
This query calculates the average year of publication for each book title, and returns only the titles with an average year greater than 2010. The average year is calculated by taking the average of all non-NULL year values for each book title, and this average is evaluated for each row of the dataset according to the processing rules stated in [S1] and [S2]. The result-set expressions are evaluated once for each group of rows, and the HAVING clause is evaluated once for each group of rows as a boolean expression. The result of the HAVING clause is false if the group is discarded, and the group is discarded if the result of the HAVING clause is false. The result-set expressions are then evaluated once for each group of rows, and the result is a single row of data for each group. The AVG function returns the average value of all non-NULL year values within a group, and the result is computed as total() / count() so all of the constraints that apply to total() also apply to AVG. [S3]
Judge: The explanation is correct, complete, and clear, and all claims are grounded in the SQL and schema.

---

q029 · INNER JOIN of two tables

```
SELECT patients.id, doctors.id, appointments.appt_date, patients.name AS patient_name, doctors.name AS doctor_name FROM patients INNER JOIN appointments ON patients.id = appointments.patient_id INNER JOIN doctors ON appointments.doctor_id = doctors.id;
```

Baseline: good
This query retrieves a list of patients and their corresponding appointments, along with the doctor's name and appointment date. The query starts by selecting the patient's ID and the doctor's ID, as well as the appointment date. It then joins the patients table with the appointments table on the patient's ID, and the appointments table with the doctors table on the doctor's ID. The result is a list of patients with their corresponding appointments, including the doctor's name and appointment date. The query does not include any filters or grouping, and it returns all rows from the joined tables. The result set includes the patient's ID, the doctor's ID, the appointment date, the patient's name, and the doctor's name, with the patient's name and doctor's name aliased as patient_name and doctor_name, respectively.
Judge: The explanation accurately covers all operations in the query and is clear for a non-technical reader. The only minor suggestion for improvement is to explicitly mention that the query returns the patient's name and doctor's name as aliases, which is already stated but could be more explicitly clear.

RAG: bad
This query returns the patient's ID, doctor's ID, appointment date, patient's name, and doctor's name. The query joins the patients table with the appointments table on the patient's ID, and then joins the appointments table with the doctors table on the doctor's ID. The result is the cartesian product of the patients and doctors datasets, with the appointment date and names added. [S1] The query uses the standard join syntax, but SQLite allows for non-standard join types, such as 'OUTER LEFT NATURAL JOIN'. [S5] The query does not use any non-standard join types. [S2] The query uses the 'INNER JOIN' syntax, which is allowed in SQLite. [S3] The query uses the cartesian product of the patients and doctors datasets, which is the default behavior for joins in SQLite. [S4] The query uses the 'WHERE' clause to filter out rows with NULL values, which is the standard behavior in SQLite. [S6] The query does not use any 'FULL JOIN' or 'FULL OUTER JOIN' syntax, which is allowed in SQLite. [S3] The query uses the cartesian product of the patients and doctors datasets, which is the default behavior for joins in SQLite. [S4] The query uses the 'WHERE' clause to filter out rows with NULL values, which is the standard behavior in SQLite. [S5] The query uses the standard join syntax, but SQLite allows for non-standard join types, such as 'OUTER LEFT NATURAL JOIN'. [S1] The query uses the standard join syntax, but SQLite allows for non-standard join types, such as 'OUTER LEFT NATURAL JOIN'. [S2] The query uses the standard join syntax, which is allowed in SQLite. [S3] The query uses the cartesian product of the patients and doctors datasets, which is the default behavior for joins in SQLite. [S4] The query uses the 'WHERE' clause to filter out rows with NULL values, which is the standard behavior in SQLite. [S6] The query does not use any 'FULL JOIN' or 'FULL OUTER JOIN' syntax, which is allowed in SQLite.
Judge: The explanation contains multiple unsupported claims, such as the use of non-standard join types, the use of the cartesian product, and the filtering of NULL values. The query does not use any of these features, and the schema does not support them. The explanation also describes the result as a cartesian product, which is incorrect. The query returns specific columns and joins, not a cartesian product of the datasets.

---

q045 · subquery in WHERE

```
SELECT * FROM books WHERE author IN (SELECT name FROM members WHERE id = 1);
```

Baseline: bad
This query retrieves all columns from the books table where the author is the same as the person who has a membership with id 1. In other words, it finds all books written by the person who joined the library first. The subquery first finds the name of this person, and then the main query uses this name to find the corresponding books.
Judge: The explanation incorrectly states that the subquery finds the name of the person who joined the library first, and that the main query uses this name to find books. The subquery actually finds the name of the member with id 1, and the main query filters books by this author's name.

RAG: good
The query selects all columns from the books table where the author is the same as the name of the member with id 1. This is done by using the IN operator with a subquery that returns the name of the member with id 1. The subquery is evaluated for each row in the books table, and if the author matches, the row is included in the result. [S2]
Judge: The explanation is mostly correct but omits the join operation and the subquery's role in filtering the books table.

---

q068 · LEFT JOIN with IS NULL filter
```
SELECT * FROM members LEFT JOIN loans ON members.id = loans.member_id WHERE loans.book_id IS NULL;
```

Baseline: good
This query retrieves all members who have never borrowed a book. It starts by selecting all columns from the members table. Then, it joins the members table with the loans table on the condition that the member's ID matches the loan's member ID. The LEFT JOIN ensures that all members are included, even if they don't have a matching loan record. The WHERE clause filters the results to only include loans where the book ID is NULL, meaning the member has not borrowed a book. This effectively selects all members who have never borrowed a book, along with any additional information from the members table.
Judge: The explanation is correct and complete, and the operations are accurately described. The only minor issue is that the query returns all columns from the members table, not just the members who have never borrowed a book.

RAG: bad
This query retrieves all members who have never borrowed a book. It joins the members table with the loans table on the condition that the book_id is NULL, meaning the member has not borrowed a book. The LEFT JOIN ensures that all members are included, even if they have not borrowed a book, and the WHERE clause filters out members who have borrowed a book. [S1] [S2]
Judge: The explanation incorrectly states that the query joins on book_id being NULL, which is not the case. The query filters members who have not borrowed a book, but the join condition is on member_id, not book_id. Additionally, the query does not return the correct result, as it should filter members who have no loans, not just those who have not borrowed a book.

---

q071 · subquery in WHERE
```
SELECT name FROM patients WHERE id = (SELECT patient_id FROM appointments WHERE id = 1);
```

Baseline: bad
This query finds the name of a patient who has an appointment with a specific doctor. The query first finds the patient ID of the patient who has the first appointment, and then uses that ID to find the name of the patient in the patients table. The result is the name of the patient who has the first appointment with a specific doctor.
Judge: The explanation is incorrect because the query does not find a patient based on a specific doctor, and it does not return the name of the patient who has the first appointment. It returns the name of the patient who has the appointment with the first ID in the appointments table.

RAG: good
This query returns the name of a patient who has an appointment with a doctor. The query first finds the patient_id of the first appointment (id = 1), and then uses this patient_id to find the name of the corresponding patient. The result is a single column, so it can be used as an operand of a comparison operator or as the value in an UPDATE SET clause. [S1]
Judge: The explanation is correct and complete, but could be more clear in translating SQL terms to plain language.

---

q089 · GROUP BY with HAVING
```
SELECT course_id, COUNT(*) FROM enrollments GROUP BY course_id HAVING COUNT(*) % 2 = 0;
```

Baseline: bad
This query selects the course IDs of students who have enrolled in an even number of courses. It does this by counting the number of courses each student is enrolled in, grouping the results by course ID, and then filtering the groups to only include those with an even count of courses. The exact course IDs returned are the ones that meet this condition, which is why the query uses the `COUNT(*)` function to count the number of courses each student is enrolled in.
Judge: The explanation incorrectly states that the query selects course IDs of students, when it actually selects course IDs and counts the number of enrollments. The query does not filter by students but by courses with an even number of enrollments.

RAG: good
The query selects the course_id of courses that have an even number of enrollments. The COUNT(*) expression is evaluated for each group of rows, and the HAVING clause filters out groups where the count is odd. [S1] [S2]
Judge: The explanation accurately covers all aspects of the query and is clear to a non-technical reader.

---

q092 · INNER JOIN of two tables
```
SELECT customers.name, orders.quantity, orders.order_date FROM customers INNER JOIN orders ON
customers.id = orders.customer_id;
```

Baseline: good
This query selects the customer's name, the quantity of products ordered, and the date of the order from the customers and orders tables. It joins the two tables based on the matching customer ID, and only includes rows where the customer ID exists in both tables. The result set will contain the customer's name, the quantity of products ordered, and the date of the order for each customer who has placed an order.
Judge: The explanation accurately covers all operations in the query and is clear for a non-technical reader. It is fully grounded in the provided schema.

RAG: bad
This query returns the customer's name, the quantity of the order, and the order date. The query joins the customers table with the orders table on the customer's ID, and only includes rows where the quantity is not NULL. [S3] The extra NULL rows for non-matched rows on the other operand are added after ON clause processing but before WHERE clause processing, so the ON clause does not affect the inclusion of NULL rows. [S3] The query uses the INNER JOIN operator, which is equivalent to a comma-join or CROSS JOIN without an ON clause. [S2] The result is the cartesian product of the customers and orders tables, excluding rows where the quantity is NULL. [S4]
Judge: The explanation incorrectly states that the query includes only rows where the quantity is not NULL, and it incorrectly claims that INNER JOIN is equivalent to a CROSS JOIN without an ON clause. The query does not filter on the quantity, and INNER JOIN does not produce a cartesian product.

---

q096 · INNER JOIN of two tables

```
SELECT orders.id, customers.name, products.name, orders.quantity, orders.order_date FROM orders INNER JOIN customers ON orders.customer_id = customers.id INNER JOIN products ON orders.product_id = products.id;
```

Baseline: good
This query retrieves information from three tables: customers, products, and orders. It selects specific columns from each table: the order ID, customer name, product name, order quantity, and order date. The query joins the orders table with the customers table on the customer ID, and then joins the result with the products table on the product ID. The result set includes all rows from the orders table, along with the matching customer and product information. The query does not filter or limit the number of rows returned, so it will include all orders, customers, and products that are associated with each order.
Judge: The explanation accurately covers all operations in the query and is clear for a non-technical reader. It does not include unsupported claims.

RAG: bad
This query returns all columns from the orders table, along with the customer's name and product's name, and the quantity and order date of each order. The query joins the orders table with the customers table on the customer's id, and the products table on the product's id. The result is the cartesian product of the left and right-hand datasets, with no filtering or aggregation. [S2] SQLite is not nearly so fussy about join syntax as standard SQL, and allows the use of an ON or USING clause on a comma-join or CROSS JOIN, and will omit the ON or USING clause from any join at all. [S3] All joins in SQLite are based on the cartesian product of the left and right-hand datasets, and the result is simply the cartesian product of the left and right-hand datasets if there is no ON or USING clause. [S4] If a WHERE clause is specified, the WHERE expression is evaluated for each row in the input data as a boolean expression, and only rows for which the WHERE clause expression evaluates to true are included from the dataset. [S4] In this case, there is no WHERE clause, so all rows from the orders table are included. [S5] The query uses the standard JOIN syntax, but allows for non-standard join types such as OUTER LEFT NATURAL JOIN, which is equivalent to NATURAL LEFT OUTER JOIN. [S6] The result of the query includes extra rows with NULL values in the columns that would normally contain values copied from the left-hand input dataset.
Judge: The explanation incorrectly states that the result is the cartesian product of the left and right-hand datasets, and that there is no filtering or aggregation. It also contains unsupported claims about non-standard join types and NULL values in the result.

---

q097 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT patients.name, doctors.name, appointments.appt_date FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 1;
```

Baseline: good
This SQL query retrieves the names of patients, doctors, and appointment dates from the database. It starts by joining the patients, appointments, and doctors tables based on their matching IDs. The query then orders the results by the appointment date in ascending order, so the earliest appointment date appears first. Finally, it limits the results to a single row, effectively showing the earliest appointment date and the corresponding patient and doctor names.
Judge: The explanation accurately covers all operations in the query and is clear for a non-technical reader.

RAG: bad
This query returns the names of patients, doctors, and the date of their appointments, sorted by appointment date, and limited to one row. The query joins the patients, appointments, and doctors tables based on their IDs, and filters out rows where the appointment date is not specified. The result is ordered by appointment date, with earlier dates appearing first, and NULL values are considered smaller than any other values. The query uses a LIMIT clause to return only one row, and an OFFSET clause to skip the first row. [S3] [S5]
Judge: The explanation incorrectly states that the query filters out rows where the appointment date is not specified, and it mentions an OFFSET clause which is not in the SQL. The query does not use OFFSET, and the filtering is done by the join and the ORDER BY clause.